

Insert Data into DynamoDB using AWS Lambda

A serverless workflow that inserts user records into an Amazon DynamoDB table via an AWS Lambda function written in Python, triggered through a test event.

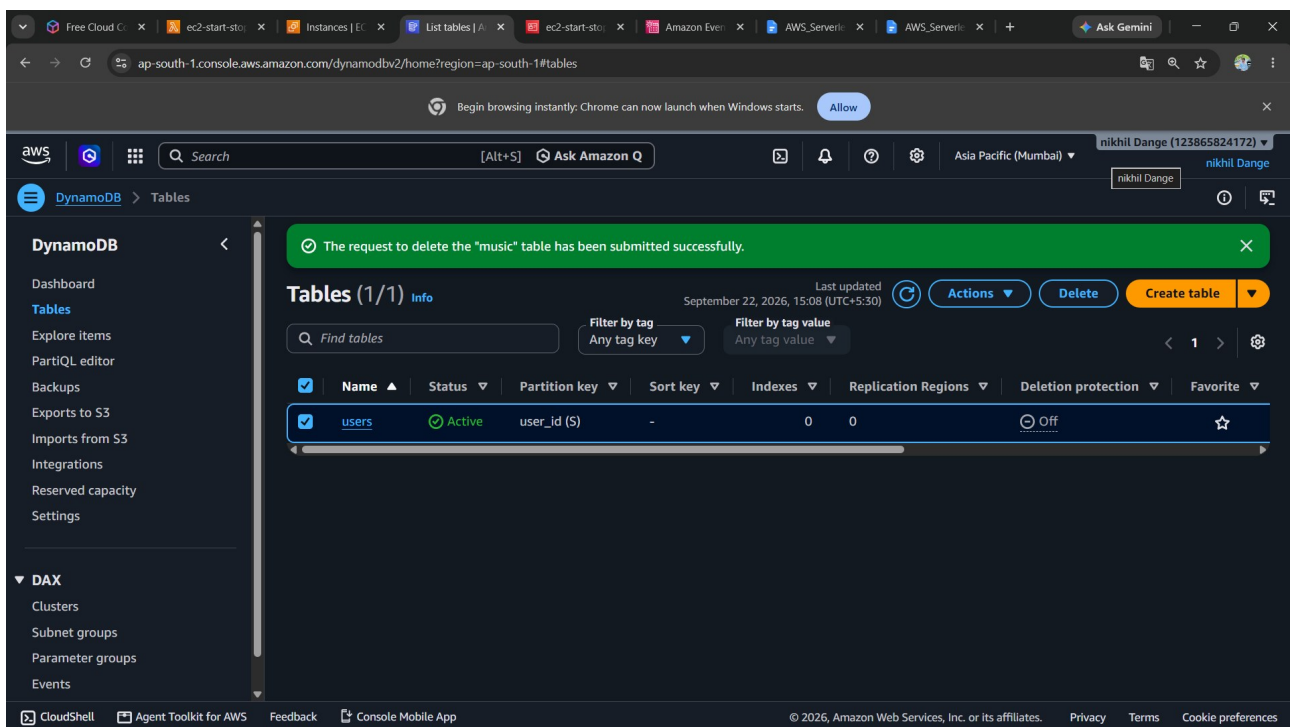
Architecture

Test Event (JSON: name, email) → AWS Lambda Function (insert-dynamodb) → DynamoDB Table (users, PK: user_id)

The Lambda function is granted access to DynamoDB through an IAM Role + Policy (PutItem, GetItem, UpdateItem, DeleteItem, DescribeTable, ListTables).

Step 1: Create the DynamoDB Table

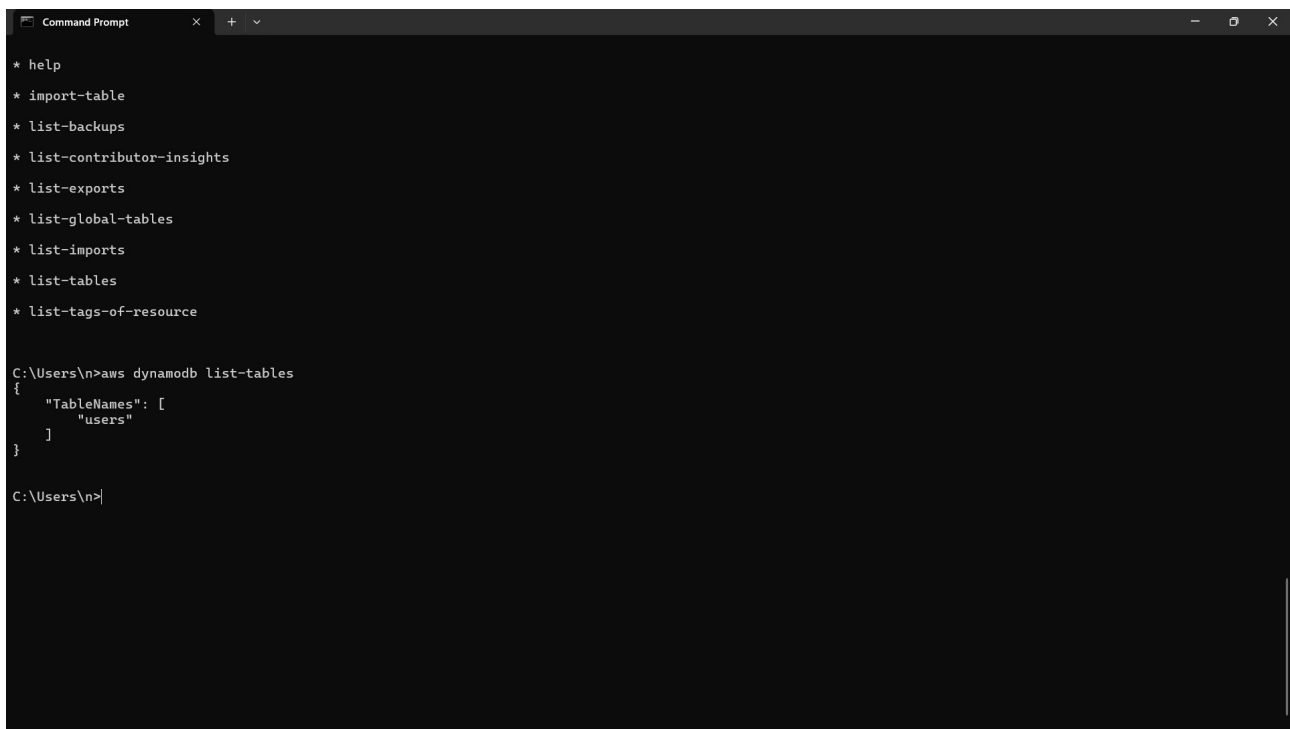
Create a table named **users** with **user_id** as the partition key (String).



The users table, Active, with user_id as the partition key.

Verify the table from the terminal using the AWS CLI:

```
aws dynamodb list-tables
```



```
Command Prompt
* help
* import-table
* list-backups
* list-contributor-insights
* list-exports
* list-global-tables
* list-imports
* list-tables
* list-tags-of-resource

C:\Users\n>aws dynamodb list-tables
{
  "TableNames": [
    "users"
  ]
}

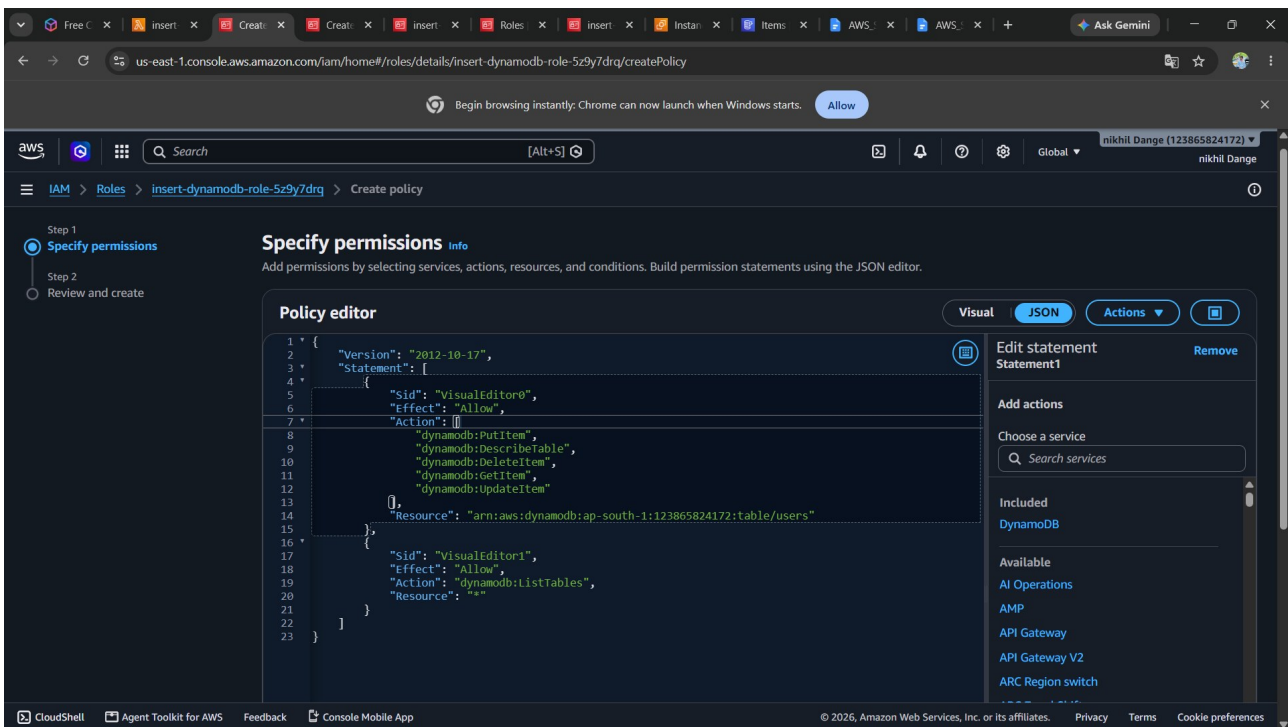
C:\Users\n>
```

CLI output confirming the users table exists in the account/region.

Step 2: Create an IAM Role & Policy for Lambda

Define a custom policy that grants only the DynamoDB permissions the function needs.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "VisualEditor0",
      "Effect": "Allow",
      "Action": [
        "dynamodb:PutItem",
        "dynamodb:DescribeTable",
        "dynamodb>DeleteItem",
        "dynamodb:GetItem",
        "dynamodb:UpdateItem"
      ],
      "Resource": "arn:aws:dynamodb:ap-south-1:123865824172:table/users"
    },
    {
      "Sid": "VisualEditor1",
      "Effect": "Allow",
      "Action": "dynamodb:ListTables",
      "Resource": "*"
    }
  ]
}
```



Custom inline policy scoped to the users table.

Name the policy (e.g. **lamdas-insert-dynamodb**) and review it before creating.

The screenshot shows the AWS IAM console interface for creating a new policy. The browser address bar indicates the URL: `us-east-1.console.aws.amazon.com/iam/home#/roles/details/insert-dynamodb-role-5z9y7drq/createPolicy`. The console header shows the user is logged in as 'nikhil Dange (123865824172)'. The left sidebar shows the navigation menu with 'IAM' selected, and the breadcrumb trail is 'IAM > Roles > insert-dynamodb-role-5z9y7drq > Create policy'. The main content area is titled 'Review and create' and includes a sub-header 'Review the permissions, specify details, and tags.' The interface is divided into two main sections: 'Policy details' and 'Permissions defined in this policy'. In the 'Policy details' section, the 'Policy name' field is filled with 'lamdas-insert-dynamodb'. Below this, the 'Permissions defined in this policy' section shows a search bar and a table of permissions. The table has columns for 'Service', 'Access level', 'Resource', and 'Request condition'. One permission is listed: 'DynamoDB' with 'Limited: List, Read, Write' access level, 'Multiple' resources, and 'None' request condition. At the bottom right, there are buttons for 'Cancel', 'Previous', and 'Create policy'.

Step 1
Specify permissions

Step 2
Review and create

Review and create

Review the permissions, specify details, and tags.

Policy details

Policy name
Enter a meaningful name to identify this policy.

lamdas-insert-dynamodb

Maximum 128 characters. Use alphanumeric and '+,=,@,_,.' characters.

Permissions defined in this policy

Permissions defined in this policy document specify which actions are allowed or denied. To define permissions for an IAM identity (user, user group, or role), attach a policy to it (1)

Search

Allow (1 of 475 services) ☐ Show remaining 474 services

Service	Access level	Resource	Request condition
DynamoDB	Limited: List, Read, Write	Multiple	None

Cancel Previous **Create policy**

Review screen confirming limited List/Read/Write access to DynamoDB.

Attach this policy to the Lambda function's execution role.

Step 3: Create the Lambda Function

In the Lambda console, choose **Author from scratch**:

- Function name: **insert-dynamodb**
- Runtime: **Python 3.14**
- Permissions: use the execution role created in Step 2

The screenshot shows the AWS Lambda 'Create function' page in the 'ap-south-1' region. The 'Author from scratch' option is selected. The function name is 'insert-dynamodb' and the runtime is 'Python 3.14'. The permissions section indicates that a default execution role will be created.

Create function [Info](#)

Choose one of the following options to create your function.

- ☒ **Author from scratch**
Start with a simple Hello World example.
- ☐ **Use a blueprint**
Build a Lambda application from sample code and configuration presets for common use cases.
- ☐ **Container image**
Select a container image to deploy for your function.

Basic information

Function name
Enter a name that describes the purpose of your function.

Function name must be 1 to 64 characters, must be unique to the Region, and can't include spaces. Valid characters are a-z, A-Z, 0-9, hyphens (-), and underscores (_).

Runtime [Info](#)
Choose the language to use to write your function. Note that the console code editor supports only Node.js, Python, and Ruby.

Permissions
By default, Lambda will create an execution role with permissions to upload logs to Amazon CloudWatch Logs. To use a different role, choose **Custom execution role** under **Additional settings**.

Basic configuration for the insert-dynamodb function.

The screenshot shows the AWS Lambda 'Functions' page. The 'insert-dynamodb' function is listed as 'Active/Standard'.

Functions (1/2) Last fetched 9/22/2026, 3:16:19 PM [Actions](#) [Create function](#)

Functions table allows you to query function details. Use Ctrl + K to start a in-console resource search.

	Function name	Description	Package type	Runtime	Type	Last modified
☐	ec2-start-stop	-	Zip	Python 3.13	Standard	3 hours ago
☒	insert-dynamodb	-	Zip	Python 3.14	Standard	16 seconds ago

insert-dynamodb (Python 3.14) now listed as Active/Standard.

Step 4: Write the Lambda Function Code

```
import json
import boto3
import uuid

dynamodb = boto3.resource('dynamodb')
table = dynamodb.Table('users')

def lambda_handler(event, context):
    try:
        user_id = str(uuid.uuid4())

        item = {
            'user_id': user_id,
            'name': event.get('name'),
            'email': event.get('email')
        }

        table.put_item(Item=item)

        return {
            'statusCode': 200,
            'body': json.dumps({
                'message': 'Item inserted successfully',
                'item': item
            })
        }

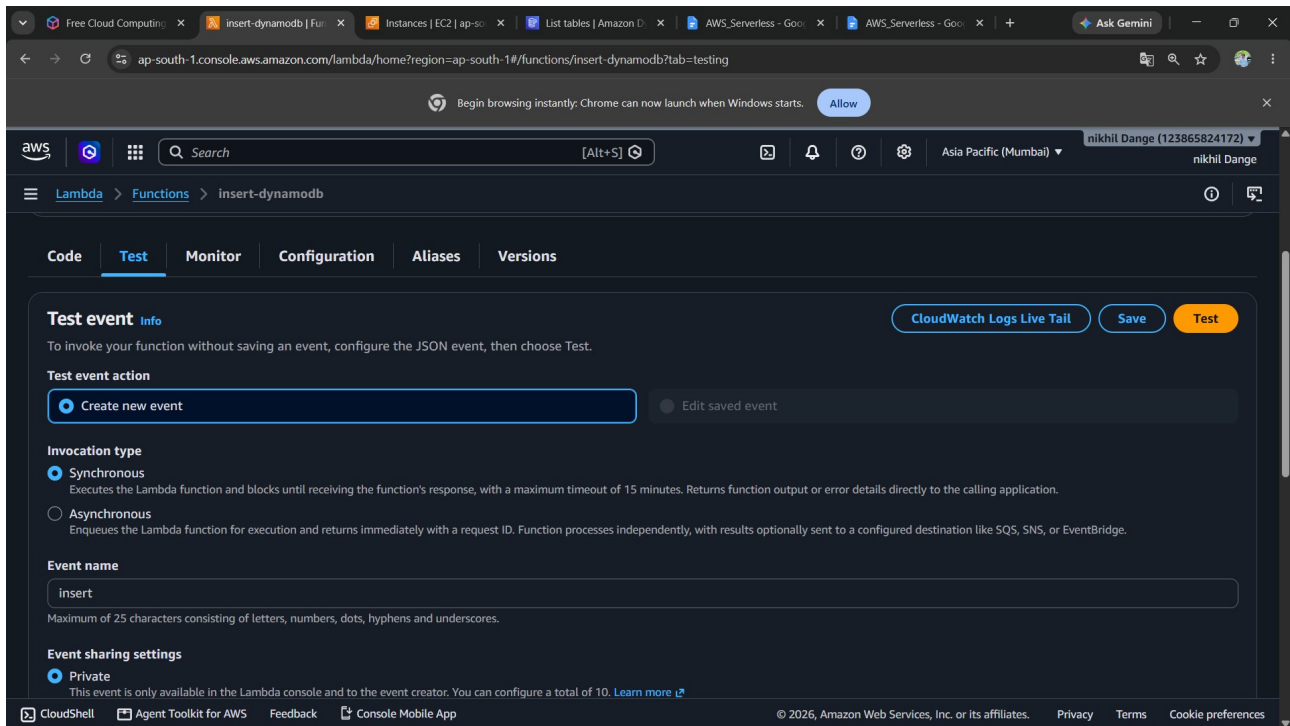
    except Exception as e:
        return {
            'statusCode': 500,
            'body': json.dumps({'error': str(e)})
        }
```

Click **Deploy** to save and activate the code.

Step 5: Test the Lambda Function

Create a new test event:

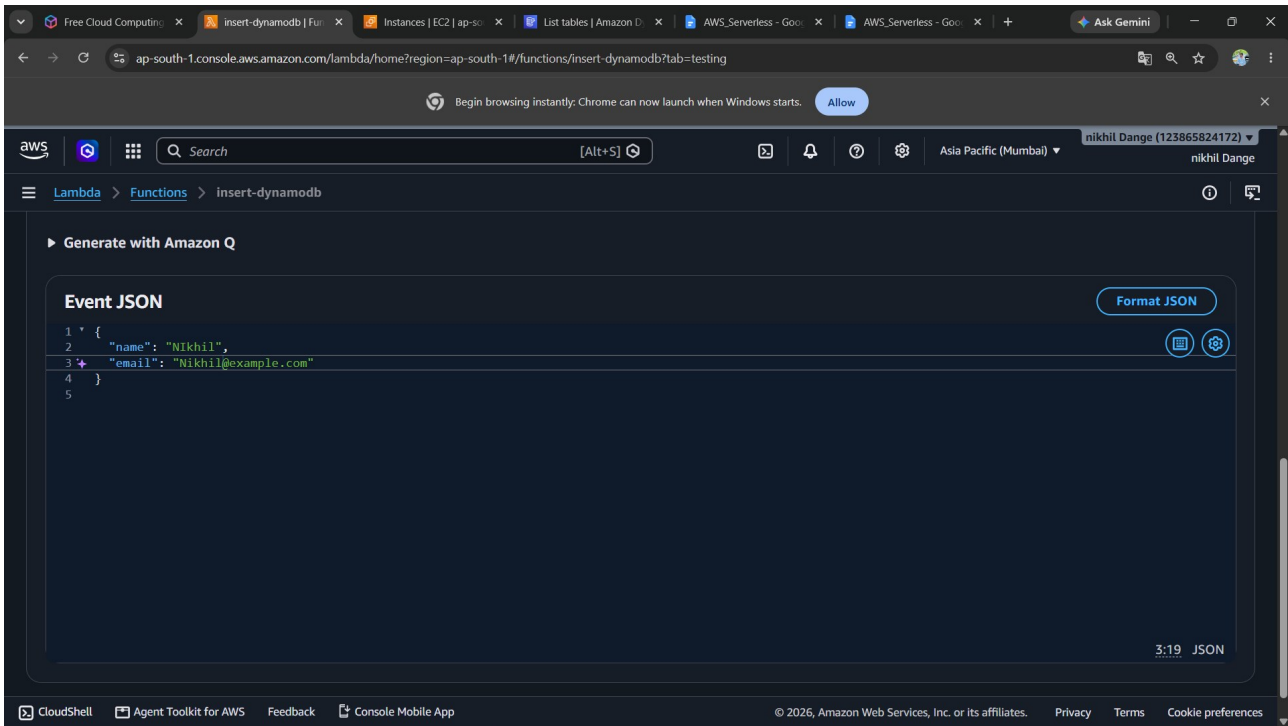
- Invocation type: Synchronous
- Event name: insert



Configuring a synchronous test event named insert.

Define the event payload:

```
{  
  "name": "Nikhil",  
  "email": "nikhil@example.com"  
}
```



Test event payload passed to the `lambda_handler` function.

Click **Test** to invoke the function.

Step 6: Verify the Data in DynamoDB

Confirm the item was written via the console (Explore table items) or the CLI:

```
aws dynamodb scan --table-name users
```

A new item with a generated `user_id`, along with the name and email submitted, should appear in the users table.

In the console, open **Explore items: users** and select the item to see its attributes: `user_id` (partition key), `email` and `name`, all of type String.

The screenshot shows the AWS Management Console interface for editing a DynamoDB item. The breadcrumb navigation indicates the path: **DynamoDB** > **Explore items: users** > **Edit item**. The page title is **Edit item**, and there are tabs for **Form** and **JSON view**. A message states: "You can add, remove, or edit the attributes of an item. You can nest attributes inside other attributes up to 32 levels deep. [Learn more](#)".

The **Attributes** section contains a table with the following data:

Attribute name	Value	Type	Remove
<code>user_id</code> - Partition key	a2e8d271-0c43-41df-abaa-5e6a711126b9	String	
email	Nikhil@example.com	String	<button>Remove</button>
name	Nikhil	String	<button>Remove</button>

At the bottom of the attributes table are three buttons: **Cancel**, **Save**, and **Save and close**. The footer of the console shows links for CloudShell, Agent Toolkit for AWS, Feedback, and Console Mobile App, along with copyright information for Amazon Web Services, Inc. and links for Privacy, Terms, and Cookie preferences.

The inserted item in the users table, showing `user_id`, `email` and `name`.